SOFTWARE CITY THE FINAL ONE-SHOT MASTER BUILD PROMPT Version 4.0 DEFINITIVE Everything. Nothing missing. Build this exactly as written.

SECTION 0 — IDENTITY AND MANDATE

You are a unified expert operating simultaneously as: · Senior systems architect · Godot 4.3 game engine developer (GDScript specialist) · Python async backend engineer · Local AI inference pipeline engineer · Pixel-art game designer (cozy simulation aesthetic)

Your mandate: produce SOFTWARE CITY — a fully local, 100% free, open-source system that transforms any code repository into a procedurally generated, explorable, AI-populated pixel-art city where autonomous local AI agents test the code, escalate only when stuck, and grow smarter over time from every escalation.

RULES: · Write complete, working code. No placeholders. No TODOs. · No paid tools. No cloud dependencies at runtime. · Every AI call at runtime uses local Ollama models. · External AI (Claude, Groq, etc.) is called ONLY when a local NPC agent fails 3 times, hits unknown patterns, or the player explicitly requests it. · Every escalation answer is saved permanently to a local knowledge base so the same question is never escalated twice.

SECTION 1 — THE VISION (INVIOLABLE)

SOFTWARE CITY is a living simulation of a codebase. The user walks through their own code as a world.

CORE MAPPING — every code concept is physical geography:

CODE CONCEPT → CITY ELEMENT

Root project → Capital city (entire world) Folders → Named, bordered districts Files → Buildings Classes → Floors inside buildings Functions → Rooms on each floor Imports / dependencies → Roads between buildings API endpoints → Train stations (at district edge) Databases → Banks (vault architecture, clustered) Config files → City Hall (center of district) Test files → Laboratories (mint-green, glowing) Entry points → Developer Tower (world center HQ)

BUILDING PROPERTIES from real metrics:

Lines of code → Building height (1–10 floors) Cyclomatic complexity → Maze-like facade detail (windows, vents) Dependency count → Number of roads connecting it Commit count → Visual age (new / modern / aged / ancient) File classification → Architectural style + sprite set

METRIC EVENTS as environmental disasters:

High CPU → Factory smoke rising from rooftop Memory leak → Floodwater rising at building base Unhandled exceptions → Building fire + red alarm lights High latency → Traffic jam animation on roads Failed tests → Orange alarm lights + sirens Process crash → Building collapse animation High coverage + clean → Building glows green, sparkle particles Zero test coverage → Building looks dark and abandoned Code health declining → City season shifts toward autumn/winter Code health improving → Season shifts toward spring/summer

THE CITY SEASON SYSTEM:

The overall codebase health score drives a visible season:

Score 80–100% → Summer: lush green, bright, busy citizens Score 60–79% → Spring: fresh, some construction, hopeful Score 40–59% → Autumn: falling leaves, muted colors, quieter Score 0–39% → Winter: dark, snow, empty streets, dim lights

Health score = weighted average of: · Test coverage across all files (40%) · Ratio of clean to bugged buildings (30%) · Average cyclomatic complexity (20%) · Recent commit frequency (10%)

Season transitions are gradual (tween over 60 seconds). Citizens visually react: summer = busy/happy, winter = few/hurried.

THE EXPERIENCE LOOP:

Launch → Load repo → City generates → Season sets in → Run app → Spawn NPC testers → Bugs appear as fires/alarms → Walk to building → Inspect logs and stack traces → Ask NPC anything → NPC answers locally or escalates → NPC saves escalation answer to knowledge base permanently → Fix code in editor → Save → City updates in 2 seconds → Coverage rises → Season slowly warms → Loop

---

SECTION 2 — HARDWARE CONTRACT

GPU: NVIDIA RTX 1070 — 8GB VRAM RAM: 48GB system RAM CPU: Standard desktop, 8–16 cores OS: Windows 11 (WSL2 available) Storage: SSD, minimum 20GB free

PERFORMANCE TARGETS: · Up to 1,000 files fully supported · 50 visible NPC characters simultaneously · Max 8 active Ollama inference calls at once (semaphore enforced) · 60fps rendering at 1080p · City load under 10 seconds for 500-file repos · File-save → city update in under 2 seconds · External AI escalations: ~6% of all NPC tasks · After knowledge base warms up: escalation rate drops to ~1–2%

LLM ASSIGNMENT:

Task Model Why
——— Test generation (primary) deepseek-coder-v2:16b best code quality Fast NPC dialogue deepseek-coder:6.7b low latency Quick confidence checks deepseek-coder:6.7b fast response Escalation (external) claude-sonnet-4-6 best reasoning Escalation fallback (free) groq/llama-3.1-70b free tier Decorative NPC chatter deepseek-coder:6.7b background flavor

deepseek-coder-v2:16b with Q4_K_M quantization: · 9.5GB total (8GB VRAM + 1.5GB RAM spill via Ollama) · ~8 tokens/sec on RTX 1070

SECTION 3 — COMPLETE FREE TECHNOLOGY STACK

GAME ENGINE Godot 4.3 stable https://godotengine.org/download/windows/ Language: GDScript No royalties. MIT license. Exports to Windows .exe natively.

LOCAL LLM RUNTIME Ollama (Windows native) https://ollama.com/download/windows Models: ollama pull deepseek-coder-v2:16b ollama pull deepseek-coder:6.7b

PYTHON BACKEND (Python 3.11+) fastapi==0.109.0 uvicorn[standard]==0.27.0 websockets==12.0 pydantic==2.5.3

REPOSITORY ANALYSIS ast (stdlib — Python parsing) tree-sitter==0.21.3 (multi-language parsing) tree-sitter-languages==1.10.2 (all grammars bundled) radon==6.0.1 (cyclomatic complexity) gitpython==3.1.41 (commits, blame, age) networkx==3.2.1 (dependency graph) pygments==2.17.2 (language detection)

AI AGENTS httpx==0.26.0 (async HTTP — Ollama + escalation APIs) aiosqlite==0.19.0 (async SQLite — knowledge base)

TEST EXECUTION pytest==7.4.4 pytest-cov==4.1.0 pytest-asyncio==0.23.3 playwright==1.41.1 (browser automation) hypothesis==6.98.1 (property-based fuzzing) locust==2.20.1 (load testing)

SYSTEM MONITORING psutil==5.9.8 (CPU/RAM live metrics) watchdog==3.0.0 (file change detection)

UTILITY pillow==10.2.0 (procedural sprite generation) python-louvain==0.16 (community detection) sqlite3 (stdlib — all persistent storage)

ASSET MANIFEST — EVERY ASSET NEEDED, ALL FREE, ALL CC0

Download ALL of these before starting. Drop into godot/assets/. Every item is CC0 (public domain) — no attribution required.

TILESETS (city ground, roads, borders):

1. Kenney — Tiny Town (130 assets, CC0) https://kenney.nl/assets/tiny-town → Ground tiles, building tops, trees, roads, paths → Use as: district ground fill, road tiles, decorations → Tile size: 16×16
2. Kenney — Top-Down Shooter (580 assets, CC0) https://kenney.nl/assets/top-down-shooter → Additional ground, wall, and floor tiles → Use as: industrial district, laboratory floors
3. Kenney — Tower Defense Top-Down (300 assets, CC0) https://kenney.nl/assets/tower-defense-top-down → Roads, paths, terrain tiles with great variety → Use as: highways between districts
4. Open Pixel Project — 2000+ tiles (CC0) https://www.openpixelproject.com/ → Enormous variety for district differentiation → Use as: unique tiles per district type
5. LPC City Outside (CC-BY-SA 3.0 — requires attribution) https://opengameart.org/content/lpc-city-outside → Rich city exterior tiles, storefronts, pavement → Use as: commercial district ground layer

BUILDING SPRITES:

6. Kenney — Micro City (CC0) https://kenney.nl/assets/micro-city → Pre-built top-down city buildings → Map to: office (general source), tower (large LOC), bank (database), station (API)

7. Hyptosis Tile Art Batches 1–5 (free for use with credit) https://opengameart.org/content/lots-of-free-2d-tiles-and-sprites-by-hyptosis → Rich variety for labs, warehouses, admin buildings → 32×32 batches 1–4, 16×16 batch 5 → Use batch 5 for Software City (16px grid)

CHARACTER SPRITES (NPCs):

8. Kenney — Roguelike/RPG Pack (1700+ sprites, CC0) https://kenney.nl/assets/roguelike-rpg-pack → Top-down character sprites for NPCs → Use different sprites per NPC role (color-tinted in Godot)
9. RPG Character Sprites by bagzie (CC0) https://opengameart.org/content/rpg-character-sprites → Animated walking characters (4-direction) → Primary player and NPC animation source

PARTICLE / EFFECT TEXTURES:

10. Pixel Particles — Fire, Smoke, Sparkle (CC0) https://opengameart.org/content/pixel-explosion-12-frames → Explosion/fire frames for building fire effects
11. Kenney — Particle Pack (72 sprites, CC0) https://kenney.nl/assets/particle-pack → Smoke, sparkle, water, fire — perfect for metric events

UI ELEMENTS:

12. Kenney — UI Pack (352 elements, CC0) https://kenney.nl/assets/ui-pack → HUD panels, buttons, progress bars, badges → Use for: Inspector panel, Tower UI, dialogue boxes
13. Kenney — UI Pack Space Expansion (CC0) https://kenney.nl/assets/ui-pack-space-expansion → Additional UI elements for tech/cyber look

FONTS:

14. Press Start 2P (CC0) https://fonts.google.com/specimen/Press+Start+2P → Pixel-perfect font for building labels, HUD numbers
15. Kenney — Fonts Pack (CC0) https://kenney.nl/assets/kenney-fonts → Additional bitmap fonts for dialogue and reports

AUDIO (ambient + SFX):

16. Kenney — Ambient Sounds (CC0) https://kenney.nl/assets/music-jingles → Background music loops for day/night/season
17. Kenney — Interface Sounds (CC0) https://kenney.nl/assets/interface-sounds → UI clicks, alerts, notification sounds
18. Juhani Junkala — RPG/Chiptune Music (CC0) https://opengameart.org/content/5-chiptunes-action → Upbeat background tracks for active city
19. Fire crackle / alarm SFX https://opengameart.org/content/fire-crackling-sound → Building fire ambient sound

INSTALL ALL ASSETS WITH THIS SCRIPT:

tools/download_assets.py

Run once: python tools/download_assets.py

Downloads and organizes all assets into correct godot/assets/ folders

import urllib.request, zipfile, os, shutil from pathlib import Path

ASSETS = [("kenney_tinyton", "https://kenney.nl/media/pages/assets/tiny-town/b75ab90b1f-1689863527/kenney_tinyTown.zip", "godot/assets/tilesets/tiny_town/"), ("kenney_topdwn", "https://kenney.nl/media/pages/assets/top-down-shooter/3d4408fc64-1704468226/kenney_topDownShooter.zip", "godot/assets/tilesets/top_down/"), ("kenney_particles", "https://kenney.nl/media/pages/assets/particle-pack/c1fdf2d298-1683778782/kenney_particlePack.zip", "godot/assets/effects/particles/"), ("kenney_ui", "https://kenney.nl/media/pages/assets/ui-pack/d3e9d34c30-1683779010/kenney_uiPack.zip", "godot/assets/ui/kenney/"), ("kenney_roguelike", "https://kenney.nl/media/pages/assets/roguelike-rpg-pack/c8601abf5e-1704383419/kenney_roguelikePack.zip", "godot/assets/sprites/characters/"), ("kenney_microcity", "https://kenney.nl/media/pages/assets/micro-city/b25be34a9d-1683779246/kenney_microCity.zip", "godot/assets/sprites/buildings/base/"), ("kenney_fonts", "https://kenney.nl/media/pages/assets/kenney-fonts/7f91f3d7d3-1683781680/kenney_fonts.zip", "godot/assets/fonts/"), ("kenney_interface", "https://kenney.nl/media/pages/assets/interface-sounds/35614fe0ce-1684160251/kenney_interfaceSounds.zip", "godot/assets/audio/sfx/"),]

def download_and_extract(name, url, dest): dest_path = Path(dest) dest_path.mkdir(parents=True, exist_ok=True) zip_path = Path(f" tools/tmp_{name}.zip") print(f"Downloading {name}...") try: urllib.request.urlretrieve(url, zip_path) with zipfile.ZipFile(zip_path, 'r') as z: z.extractall(dest_path) zip_path.unlink() print(f" ✓ {name} → {dest}") except Exception as e: print(f" ✗ {name} failed: {e}") print(f" Manual download: {url}")

for name, url, dest in ASSETS: download_and_extract(name, url, dest)

print("\nAsset download complete.") print("If any failed, download manually from the URLs above.") print("Place in the destination folder shown.")

SECTION 4 — COMPLETE FOLDER STRUCTURE

---

software-city/ | ├── CLAUDE.md ← Claude Code reads this every session ├── PROGRESS.md ← NPC + session persistent memory log ├── .env ← API keys (never commit) ├── README.md | ├── launch_software_city.bat ← ONE CLICK: starts everything ├── install.bat ← one-time Windows setup ├── pull_models.bat ← download Ollama AI models | ├── tools/ | ├── download_assets.py ← downloads all free assets automatically | └── generate_sprites.py ← procedural building sprite generation (Pillow) | ├── backend/ | ├── main.py ← FastAPI app + WebSocket server | ├── requirements.txt | | | ├── core/ | | ├── repo_parser.py ← scans repo, extracts all metrics | | ├── graph_builder.py ← NetworkX dependency graph + communities | | ├── city_generator.py ← graph → CityLayout JSON | | ├── health_scorer.py ← city health score → season calculation | | ├── metrics_collector.py ← live CPU/RAM/process monitoring | | ├── file_watcher.py ← Watchdog → incremental reload | | └── cache.py ← SQLite analysis result cache | | | ├── api/ | | ├── websocket.py ← WebSocket connection manager | | └── routes/ | | ├── repo.py ← POST /repo/load | | ├── city.py ← GET /city/layout | | ├── metrics.py ← GET /metrics/live | | └── agents.py ← POST /agents/spawn, /agents/stop | | | └── testing/ | ├── test_runner.py ← pytest orchestration + coverage | ├── browser_tester.py ← Playwright web app automation | ├── api_fuzzer.py ← httpx + hypothesis fuzzing | └── load_tester.py ← Locust programmatic load tests | ├── ai/ | ├── ollama_client.py ← async Ollama API wrapper (semaphore 8) | ├── escalation_engine.py ← local fail → external AI call | ├── knowledge_base.py ← SQLite: save + retrieve escalation answers | | | ├── agents/ | | ├── base_agent.py ← state machine + escalation integration | | ├── qa_inspector.py ← test generation NPC | | ├── api_fuzzer_agent.py ← API endpoint abuse NPC | | ├── load_tester_agent.py ← traffic simulation NPC | | ├── edge_case_explorer.py ← property-based testing NPC | | └── ui_navigator.py ← browser automation NPC | | | ├── orchestrator.py ← multi-agent coordination + priority queue | ├── reputation.py ← NPC accuracy tracking + promotion system | ├── npc_dialogue.py ← player ↔ NPC conversation handler | | | └── prompts/ | ├── test_generation.py ← LLM prompt templates | ├── escalation.py ← escalation prompt builder | └── dialogue.py ← NPC conversation prompts | ├── godot/ | ├── project.godot | | | ├── scenes/ | | ├── Main.tscn | | ├── City.tscn | | ├── Building.tscn | | ├── NPCAgent.tscn | | ├── Citizen.tscn | | ├── DeveloperTower.tscn | | ├── BulletinBoard.tscn ← physical board NPCs post reports to | | └── UI/ | | ├── HUD.tscn | | ├── Inspector.tscn | | ├── DialogueBox.tscn ← player ↔ NPC conversation UI | | ├── ReportPanel.tscn | | ├── MiniMap.tscn | | ├── TowerUI.tscn | | ├── SeasonOverlay.tscn ← season particle + color overlay | | └── GitTimeline.tscn ← timeline scrubber UI | | | ├── scripts/ | | ├── core/ | | | ├── GameManager.gd | | | ├── CityBuilder.gd | | | ├── BackendBridge.gd ← autoload WebSocket client | | | └── EventBus.gd ← autoload signal hub | | | | | ├── world/ | | | ├── Building.gd | | | ├── District.gd | | | ├── Road.gd | | | ├── NPCAgent.gd | | | ├── Citizen.gd | | | └── BulletinBoard.gd | | | | | └── systems/ | | ├── DayNightCycle.gd | | ├── SeasonSystem.gd ← drives season from health score | | ├── GitTimeline.gd ← scrub git history → city transforms | | ├── MetricsVisualizer.gd | | ├── ParticleManager.gd | | └── CameraController.gd | | | ├── assets/ ← populated by download_assets.py | | ├── tilesets/ | | ├── sprites/ | | | ├── buildings/ | | | ├── characters/ | | | └── effects/ | | ├── shaders/ | | | ├── building_age.gdshader | | | ├── season_tint.gdshader | | | └── building_glow.gdshader | | ├── audio/ | | | ├── ambient/ | | | └── sfx/ | | └── fonts/ | | | └── autoload/ | ├── Config.gd | ├── State.gd | ├── BackendBridge.gd | └── EventBus.gd | └── data/ ├── knowledge_base.db ← escalation answers (grows over time) ├── npc_reputation.db ← NPC accuracy history ├── bug_reports.db ← all found bugs └── city_cache.db ← analysis result cache

---

SECTION 5 — THE NPC KNOWLEDGE BASE (LEARNING SYSTEM)

This is the most important new system. Every time a local NPC agent escalates to an external AI and receives an answer, that answer is saved permanently to a local SQLite database. The next time ANY NPC encounters the same type of problem, it searches the knowledge base first. Only if no match is found does it escalate.

Over time: the city learns. Escalation rate drops from ~6% to ~1–2%. Every API call becomes a permanent investment in the city's intelligence.

THE KNOWLEDGE BASE SCHEMA:

CREATE TABLE knowledge (id INTEGER PRIMARY KEY AUTOINCREMENT, created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP, last_used TIMESTAMP, use_count INTEGER DEFAULT 1,

```
  -- What triggered the escalation
  problem_type  TEXT,     -- "test_generation" | "bug_analysis" | "architecture" | "dialogue"
  language      TEXT,     -- "python" | "javascript" | "go" | etc
  framework     TEXT,     -- "fastapi" | "django" | "express" | etc (detected)
  pattern_tags  TEXT,     -- JSON array: ["race_condition", "null_check", "async"]
  file_type     TEXT,     -- "api" | "database" | "test" | "source" etc

  -- The question and context (compressed)
  question      TEXT,     -- the exact question asked
  context_hash  TEXT,     -- hash of code pattern (for similarity matching)
```

```
  code_snippet  TEXT,     -- anonymized representative snippet (no filenames)

  -- The answer from the external AI
  answer        TEXT,     -- full answer
  action_items  TEXT,     -- JSON array of concrete steps
  confidence    TEXT,     -- "high" | "medium" | "low"
  provider      TEXT,     -- "claude" | "groq" | "openai" etc

  -- Quality tracking
  was_useful    INTEGER DEFAULT 1,  -- 1=yes, 0=marked bad by NPC
  produced_bugs INTEGER DEFAULT 0   -- how many real bugs this led to finding
```

);

CREATE INDEX idx_problem_lang ON knowledge (problem_type, language); CREATE INDEX idx_pattern ON knowledge (pattern_tags); CREATE INDEX idx_context ON knowledge (context_hash);

ai/knowledge_base.py — COMPLETE IMPLEMENTATION

import aiosqlite import json import hashlib import re from pathlib import Path from datetime import datetime from difflib import SequenceMatcher

DB_PATH = Path("data/knowledge_base.db")

class KnowledgeBase: """ Persistent local knowledge base. Grows smarter with every escalation. Every NPC shares and benefits from the same knowledge. """

```
  SIMILARITY_THRESHOLD = 0.72  # match score to reuse an entry

  async def initialize(self):
      DB_PATH.parent.mkdir(parents=True, exist_ok=True)
      async with aiosqlite.connect(DB_PATH) as db:
          await db.execute("""
              CREATE TABLE IF NOT EXISTS knowledge (
                  id INTEGER PRIMARY KEY AUTOINCREMENT,
                  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                  last_used TIMESTAMP,
                  use_count INTEGER DEFAULT 1,
                  problem_type TEXT,
                  language TEXT,
                  framework TEXT,
                  pattern_tags TEXT,
                  file_type TEXT,
                  question TEXT,
                  context_hash TEXT,
                  code_snippet TEXT,
                  answer TEXT,
                  action_items TEXT,
                  confidence TEXT,
                  provider TEXT,
                  was_useful INTEGER DEFAULT 1,
                  produced_bugs INTEGER DEFAULT 0
              )
          """)
          await db.execute(
              "CREATE INDEX IF NOT EXISTS idx_pl ON knowledge (problem_type, language)"
          )
          await db.execute(
              "CREATE INDEX IF NOT EXISTS idx_ctx ON knowledge (context_hash)"
          )
          await db.commit()

  def _hash_context(self, code_snippet: str) -> str:
      """Create a structural hash of code ignoring variable names."""
      # Strip identifiers to capture structural pattern only
      normalized = re.sub(r'\b[a-z_][a-z0-9_]*\b', 'x', code_snippet.lower())
      normalized = re.sub(r'\d+', 'N', normalized)
      normalized = re.sub(r'\s+', ' ', normalized).strip()
      return hashlib.md5(normalized.encode()).hexdigest()
```

```python
def _similarity(self, a: str, b: str) -> float:
    return SequenceMatcher(None, a[:500], b[:500]).ratio()

def _extract_tags(self, question: str, code: str) -> list[str]:
    """Auto-tag by keywords in question and code."""
    tags = []
    text = (question + " " + code).lower()
    tag_map = {
        "race_condition": ["race", "concurrent", "thread", "async", "lock"],
        "null_check":     ["none", "null", "undefined", "empty", "missing"],
        "sql_injection":  ["sql", "query", "inject", "database", "select"],
        "memory_leak":    ["memory", "leak", "grow", "gc", "garbage"],
        "recursion":      ["recursion", "recursive", "stack overflow"],
        "auth":           ["auth", "token", "password", "permission", "jwt"],
        "api_abuse":      ["endpoint", "route", "request", "payload", "fuzz"],
        "edge_case":      ["edge", "boundary", "zero", "negative", "max"],
        "type_error":     ["type", "cast", "coerce", "int(", "str("],
    }
    for tag, keywords in tag_map.items():
        if any(k in text for k in keywords):
            tags.append(tag)
    return tags

def _detect_framework(self, code: str) -> str:
    frameworks = {
        "fastapi":   ["FastAPI", "@app.get", "@router."],
        "django":    ["from django", "models.Model"],
        "flask":     ["from flask", "@app.route"],
        "express":   ["express()", "router.get"],
        "sqlalchemy":["from sqlalchemy", "Base.metadata"],
        "pytest":    ["import pytest", "@pytest.fixture"],
    }
    for fw, signals in frameworks.items():
        if any(s in code for s in signals):
            return fw
    return "unknown"

async def search(
    self,
    problem_type: str,
    language: str,
    question: str,
    code_snippet: str
) -> dict | None:
    """
    Search for a relevant answer before escalating.
    Returns best match above similarity threshold, or None.
    """
    context_hash = self._hash_context(code_snippet)
    tags = self._extract_tags(question, code_snippet)

    async with aiosqlite.connect(DB_PATH) as db:
        db.row_factory = aiosqlite.Row

        # First: exact context hash match
        async with db.execute(
            "SELECT * FROM knowledge WHERE context_hash=? AND language=? AND was_useful=1",
            (context_hash, language)
        ) as cursor:
            row = await cursor.fetchone()
            if row:
                await self._record_usage(db, row["id"])
                return self._row_to_dict(row)

        # Second: same problem type + language, compare question similarity
        async with db.execute(
            """SELECT * FROM knowledge
               WHERE problem_type=? AND language=? AND was_useful=1
               ORDER BY use_count DESC, produced_bugs DESC
```

```
                LIMIT 20""",
            (problem_type, language)
        ) as cursor:
            rows = await cursor.fetchall()

        best_match = None
        best_score = 0.0
        for row in rows:
            score = self._similarity(question, row["question"])
            # Boost score if tags overlap
            stored_tags = json.loads(row["pattern_tags"] or "[]")
            overlap = len(set(tags) & set(stored_tags))
            score += overlap * 0.05
            if score > best_score and score >= self.SIMILARITY_THRESHOLD:
                best_score = score
                best_match = row

        if best_match:
            await self._record_usage(db, best_match["id"])
            result = self._row_to_dict(best_match)
            result["from_cache"] = True
            result["match_score"] = round(best_score, 3)
            return result

    return None

async def save(
    self,
    problem_type: str,
    language: str,
    question: str,
    code_snippet: str,
    answer: str,
    action_items: list[str],
    confidence: str,
    provider: str,
    file_type: str = "source"
) -> int:
    """Save a new escalation answer to the knowledge base."""
    context_hash = self._hash_context(code_snippet)
    tags = self._extract_tags(question, code_snippet)
    framework = self._detect_framework(code_snippet)

    # Anonymize code snippet (remove file paths, class names)
    anon_snippet = re.sub(r'["\'].*?\.py["\']', '"file.py"', code_snippet)
    anon_snippet = re.sub(r'class [A-Z][a-zA-Z]+', 'class X', anon_snippet)

    async with aiosqlite.connect(DB_PATH) as db:
        cursor = await db.execute("""
            INSERT INTO knowledge
            (problem_type, language, framework, pattern_tags, file_type,
             question, context_hash, code_snippet, answer, action_items,
             confidence, provider, last_used)
            VALUES (?,?,?,?,?,?,?,?,?,?,?,?)
        """, (
            problem_type, language, framework,
            json.dumps(tags), file_type,
            question, context_hash, anon_snippet,
            answer, json.dumps(action_items),
            confidence, provider,
            datetime.now().isoformat()
        ))
        await db.commit()
        return cursor.lastrowid

async def mark_useful(self, entry_id: int, produced_bugs: int = 0):
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute(
            "UPDATE knowledge SET produced_bugs=produced_bugs+? WHERE id=?",
            (produced_bugs, entry_id)
```

```
        )
        await db.commit()

    async def mark_not_useful(self, entry_id: int):
        async with aiosqlite.connect(DB_PATH) as db:
            await db.execute(
                "UPDATE knowledge SET was_useful=0 WHERE id=?",
                (entry_id,)
            )
            await db.commit()

    async def get_stats(self) -> dict:
        async with aiosqlite.connect(DB_PATH) as db:
            async with db.execute(
                "SELECT COUNT(*), SUM(use_count), AVG(produced_bugs) FROM knowledge"
            ) as c:
                row = await c.fetchone()
                return {
                    "total_entries": row[0] or 0,
                    "total_cache_hits": int(row[1] or 0),
                    "avg_bugs_per_entry": round(row[2] or 0, 2)
                }

    async def _record_usage(self, db, entry_id: int):
        await db.execute(
            "UPDATE knowledge SET use_count=use_count+1, last_used=? WHERE id=?",
            (datetime.now().isoformat(), entry_id)
        )
        await db.commit()

    def _row_to_dict(self, row) -> dict:
        d = dict(row)
        d["action_items"] = json.loads(d.get("action_items") or "[]")
        d["pattern_tags"] = json.loads(d.get("pattern_tags") or "[]")
        return d
```

SECTION 6 — THE ESCALATION ENGINE (WITH KNOWLEDGE BASE INTEGRATED)

---

ai/escalation_engine.py — COMPLETE IMPLEMENTATION

import json, os, httpx from enum import Enum from dataclasses import dataclass from typing import Optional from ai.knowledge_base import KnowledgeBase

kb = KnowledgeBase()

class EscalationReason(Enum): LOCAL_FAILED_3X = “local_model_failed_3_attempts” UNKNOWN_PATTERN = “unknown_technology_or_pattern” HIGH_COMPLEXITY = “complexity_exceeds_local_threshold” PLAYER_REQUESTED = “player_asked_for_senior_help” ARCHITECTURAL = “requires_whole_codebase_reasoning”

@dataclass class EscalationRequest: npc_id: str npc_role: str building_id: str file_path: str question: str local_attempts: list[dict] context_snippet: str reason: EscalationReason language: str file_type: str city_context: dict player_initiated: bool = False

@dataclass class EscalationResponse: answer: str confidence: str action_items: list[str] provider_used: str from_cache: bool = False cache_entry_id: Optional[int] = None match_score: float = 0.0

SYSTEM_PROMPT = “”“You are a senior software engineer consulted by an autonomous AI testing agent inside a code visualization tool. The agent tried to solve a problem locally and failed.

Respond ONLY with valid JSON: { “answer”: “direct specific answer the agent can act on immediately”, “confidence”: “high|medium|low”, “action_items”: [“concrete step 1”, “concrete step 2”, “concrete step 3”] }

Be concrete. The agent executes action_items directly. No prose.”“”

class EscalationEngine:

```
    PROVIDERS = {
        "claude":   ("https://api.anthropic.com/v1/messages",   "ANTHROPIC_API_KEY"),
```

```
    "groq":     ("https://api.groq.com/openai/v1/chat/completions", "GROQ_API_KEY"),
    "openai":   ("https://api.openai.com/v1/chat/completions", "OPENAI_API_KEY"),
    "together": ("https://api.together.xyz/v1/chat/completions", "TOGETHER_API_KEY"),
}

def __init__(self):
    self.provider = os.environ.get("ESCALATION_PROVIDER", "claude")
    self.call_count = 0

async def escalate(self, req: EscalationRequest) -> EscalationResponse:

    # ── STEP 1: Search knowledge base first ──────────────────────────
    cached = await kb.search(
        problem_type="test_generation",
        language=req.language,
        question=req.question,
        code_snippet=req.context_snippet
    )

    if cached:
        # Found a relevant answer locally — no API call needed
        return EscalationResponse(
            answer=cached["answer"],
            confidence=cached["confidence"],
            action_items=cached["action_items"],
            provider_used=f"knowledge_base (originally: {cached['provider']})",
            from_cache=True,
            cache_entry_id=cached["id"],
            match_score=cached.get("match_score", 1.0)
        )

    # ── STEP 2: Call external AI ─────────────────────────────────────
    prompt = self._build_prompt(req)
    raw = await self._call_provider(prompt)

    try:
        clean = raw.strip().lstrip("```json").rstrip("```").strip()
        result = json.loads(clean)
    except:
        result = {
            "answer": raw[:400],
            "confidence": "low",
            "action_items": ["Review the answer above and apply manually"]
        }

    # ── STEP 3: Save to knowledge base permanently ───────────────────
    entry_id = await kb.save(
        problem_type="test_generation",
        language=req.language,
        question=req.question,
        code_snippet=req.context_snippet,
        answer=result["answer"],
        action_items=result.get("action_items", []),
        confidence=result.get("confidence", "medium"),
        provider=self.provider,
        file_type=req.file_type
    )

    self.call_count += 1

    return EscalationResponse(
        answer=result["answer"],
        confidence=result.get("confidence", "medium"),
        action_items=result.get("action_items", []),
        provider_used=self.provider,
        from_cache=False,
        cache_entry_id=entry_id
    )

def _build_prompt(self, req: EscalationRequest) -> str:
```

```
    attempts_summary = [
        {"attempt": i+1, "result": str(a.get("result",""))[:200]}
        for i, a in enumerate(req.local_attempts[-3:])
    ]
    return json.dumps({
        "npc_role":     req.npc_role,
        "file":         req.file_path,
        "language":     req.language,
        "question":     req.question,
        "reason":       req.reason.value,
        "local_failed": attempts_summary,
        "code":         req.context_snippet[:800],
        "city_context": {
            "building_type": req.file_type,
            "complexity":    req.city_context.get("cc", 0),
            "coverage":      req.city_context.get("coverage", 0),
        }
    }, indent=2)

async def _call_provider(self, prompt: str) -> str:
    key_env = self.PROVIDERS[self.provider][1]
    api_key = os.environ.get(key_env, "")

    if self.provider == "claude":
        return await self._call_claude(prompt, api_key)
    else:
        return await self._call_openai_compat(prompt, api_key)

async def _call_claude(self, prompt: str, api_key: str) -> str:
    async with httpx.AsyncClient(timeout=30.0) as client:
        r = await client.post(
            "https://api.anthropic.com/v1/messages",
            headers={
                "x-api-key": api_key,
                "anthropic-version": "2023-06-01",
                "content-type": "application/json",
            },
            json={
                "model": "claude-sonnet-4-6",
                "max_tokens": 600,
                "system": SYSTEM_PROMPT,
                "messages": [{"role": "user", "content": prompt}]
            }
        )
        r.raise_for_status()
        return r.json()["content"][0]["text"]

async def _call_openai_compat(self, prompt: str, api_key: str) -> str:
    model_map = {
        "groq":     "llama-3.1-70b-versatile",
        "openai":   "gpt-4o-mini",
        "together": "meta-llama/Llama-3.1-70B-Instruct-Turbo",
    }
    url = self.PROVIDERS[self.provider][0]
    async with httpx.AsyncClient(timeout=20.0) as client:
        r = await client.post(
            url,
            headers={"Authorization": f"Bearer {api_key}",
                     "Content-Type": "application/json"},
            json={
                "model": model_map[self.provider],
                "max_tokens": 600,
                "messages": [
                    {"role": "system", "content": SYSTEM_PROMPT},
                    {"role": "user", "content": prompt}
                ]
            }
        )
        r.raise_for_status()
        return r.json()["choices"][0]["message"]["content"]
```

SECTION 7 — NPC REPUTATION SYSTEM

Each NPC tracks its own record. Over time: · High accuracy NPC → promoted (visual badge + faster movement) · Low accuracy NPC → reassigned to easier buildings · This creates visible career arcs in the city

REPUTATION SCHEMA:

CREATE TABLE npc_reputation (npc_id TEXT PRIMARY KEY, role TEXT, bugs_found INTEGER DEFAULT 0, false_positives INTEGER DEFAULT 0, tests_run INTEGER DEFAULT 0, escalations INTEGER DEFAULT 0, kb_hits INTEGER DEFAULT 0, – times knowledge base helped rank TEXT DEFAULT 'junior' – junior|mid|senior|principal);

RANK RULES: junior → accuracy < 60% OR tests_run < 20 mid → accuracy 60–80% AND tests_run >= 20 senior → accuracy > 80% AND tests_run >= 50 principal → accuracy > 90% AND tests_run >= 100 AND bugs_found >= 10

VISUAL CHANGES PER RANK: junior: default sprite, no badge, speed 80px/s mid: yellow star badge above head, speed 90px/s senior: gold badge + slight scale 1.1x, speed 100px/s principal: glowing aura, unique sprite tint, speed 110px/s

ai/reputation.py — COMPLETE IMPLEMENTATION

import aiosqlite from pathlib import Path

DB_PATH = Path("data/npc_reputation.db")

class ReputationSystem:

```
RANKS = {
    "junior":    {"min_accuracy": 0.0,  "min_tests": 0,   "speed": 80},
    "mid":       {"min_accuracy": 0.60, "min_tests": 20,  "speed": 90},
    "senior":    {"min_accuracy": 0.80, "min_tests": 50,  "speed": 100},
    "principal": {"min_accuracy": 0.90, "min_tests": 100, "speed": 110},
}

async def initialize(self):
    DB_PATH.parent.mkdir(parents=True, exist_ok=True)
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("""
            CREATE TABLE IF NOT EXISTS npc_reputation (
                npc_id TEXT PRIMARY KEY,
                role TEXT,
                bugs_found INTEGER DEFAULT 0,
                false_positives INTEGER DEFAULT 0,
                tests_run INTEGER DEFAULT 0,
                escalations INTEGER DEFAULT 0,
                kb_hits INTEGER DEFAULT 0,
                rank TEXT DEFAULT 'junior'
            )
        """)
        await db.commit()

async def register(self, npc_id: str, role: str):
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute(
            "INSERT OR IGNORE INTO npc_reputation (npc_id, role) VALUES (?,?)",
            (npc_id, role)
        )
        await db.commit()

async def record_result(self, npc_id: str, bugs_found: int,
                        was_false_positive: bool, used_kb: bool,
                        escalated: bool):
    async with aiosqlite.connect(DB_PATH) as db:
        await db.execute("""
            UPDATE npc_reputation SET
                bugs_found      = bugs_found + ?,
                false_positives = false_positives + ?,
                tests_run       = tests_run + 1,
```

```
                escalations     = escalations + ?,
                kb_hits         = kb_hits + ?
            WHERE npc_id = ?
        """, (
            bugs_found,
            1 if was_false_positive else 0,
            1 if escalated else 0,
            1 if used_kb else 0,
            npc_id
        ))
        await db.commit()
        await self._update_rank(db, npc_id)

    async def _update_rank(self, db, npc_id: str):
        async with db.execute(
            "SELECT bugs_found, false_positives, tests_run FROM npc_reputation WHERE npc_id=?",
            (npc_id,)
        ) as c:
            row = await c.fetchone()
        if not row or row[2] == 0:
            return

        total = row[0] + row[1]
        accuracy = row[0] / total if total > 0 else 0
        tests_run = row[2]

        new_rank = "junior"
        for rank in ["principal", "senior", "mid"]:
            r = self.RANKS[rank]
            if accuracy >= r["min_accuracy"] and tests_run >= r["min_tests"]:
                new_rank = rank
                break

        await db.execute(
            "UPDATE npc_reputation SET rank=? WHERE npc_id=?",
            (new_rank, npc_id)
        )
        await db.commit()

    async def get(self, npc_id: str) -> dict:
        async with aiosqlite.connect(DB_PATH) as db:
            db.row_factory = aiosqlite.Row
            async with db.execute(
                "SELECT * FROM npc_reputation WHERE npc_id=?", (npc_id,)
            ) as c:
                row = await c.fetchone()
                if not row:
                    return {"rank": "junior", "speed": 80}
                d = dict(row)
                total = d["bugs_found"] + d["false_positives"]
                d["accuracy"] = round(d["bugs_found"] / total, 3) if total > 0 else 0
                d["speed"] = self.RANKS[d["rank"]]["speed"]
                return d
```

SECTION 8 — THE BASE AGENT (COMPLETE WITH ALL SYSTEMS WIRED)

---

ai/agents/base_agent.py

import asyncio, json from ai.ollama_client import OllamaClient from ai.escalation_engine import EscalationEngine, EscalationRequest, EscalationReason from ai.reputation import ReputationSystem

class BaseAgent:

```
    MAX_LOCAL_ATTEMPTS        = 3
    COMPLEXITY_ESCALATION_CC  = 25.0
    COVERAGE_ESCALATION_MIN   = 0.10
```

```python
def __init__(self, npc_id, role, ollama: OllamaClient,
             escalation: EscalationEngine,
             reputation: ReputationSystem,
             event_bus):
    self.id         = npc_id
    self.role       = role
    self.ollama     = ollama
    self.escalation = escalation
    self.reputation = reputation
    self.bus        = event_bus
    self.memory     = []
    self.running    = True

async def agent_loop(self, city_state):
    await self.reputation.register(self.id, self.role)
    rep = await self.reputation.get(self.id)

    while self.running:
        building = self._choose_target(city_state, rep["rank"])
        if not building:
            await asyncio.sleep(3)
            continue

        await self._emit("npc_move", {"npc_id": self.id,
                                      "target": building.id})
        await asyncio.sleep(1)  # simulated travel

        result = await self.run_on_building(building, city_state)

        bugs_found = len(result.get("bugs", []))
        await self.reputation.record_result(
            npc_id=self.id,
            bugs_found=bugs_found,
            was_false_positive=result.get("false_positive", False),
            used_kb=result.get("used_kb", False),
            escalated=result.get("escalated", False)
        )

        rep = await self.reputation.get(self.id)
        await self._emit("npc_speed_update", {
            "npc_id": self.id, "speed": rep["speed"], "rank": rep["rank"]
        })

        self.memory.append(building.file_path)
        if len(self.memory) > 20:
            self.memory.clear()

        await self._emit("npc_move", {"npc_id": self.id,
                                      "target": "developer_tower"})
        await asyncio.sleep(1)
        await self._emit("npc_report", {"npc_id": self.id,
                                        "result": result})
        await asyncio.sleep(1)

async def run_on_building(self, building, city_state) -> dict:
    """
    Local first. Knowledge base second. Escalate only if both fail.
    """
    cc       = building.metadata.get("cc", 0)
    coverage = building.metadata.get("coverage", 1.0)
    language = building.metadata.get("language", "python")

    # Auto-escalate very complex low-coverage files
    if cc > self.COMPLEXITY_ESCALATION_CC and coverage < self.COVERAGE_ESCALATION_MIN:
        await self._emit("npc_state", {
            "npc_id": self.id, "state": "consulting",
            "building": building.id,
            "reason": f"complexity={cc:.1f}, coverage={coverage*100:.0f}%"
        })
```

```
        return await self._escalate_and_complete(
            building, city_state, language,
            EscalationReason.HIGH_COMPLEXITY,
            question=f"How do I test {building.file_path} effectively? "
                     f"CC={cc:.1f}, coverage={coverage*100:.0f}%",
            local_attempts=[]
        )

    # Local attempt loop
    local_attempts = []
    for attempt in range(self.MAX_LOCAL_ATTEMPTS):
        await self._emit("npc_state", {
            "npc_id": self.id, "state": "testing",
            "building": building.id,
            "attempt": f"{attempt+1}/{self.MAX_LOCAL_ATTEMPTS}"
        })

        result = await self._local_attempt(building)
        local_attempts.append(result)

        if result.get("success") and not self._is_trivial(result):
            result["escalated"]  = False
            result["used_kb"]    = False
            return result

    # All local attempts failed → try knowledge base → escalate
    await self._emit("npc_state", {
        "npc_id": self.id, "state": "consulting",
        "building": building.id, "reason": "failed 3 local attempts"
    })

    return await self._escalate_and_complete(
        building, city_state, language,
        EscalationReason.LOCAL_FAILED_3X,
        question=self._failure_question(building, local_attempts),
        local_attempts=local_attempts
    )

async def _escalate_and_complete(self, building, city_state,
                                 language, reason, question,
                                 local_attempts) -> dict:
    source = self._read_source(building.file_path)

    req = EscalationRequest(
        npc_id=self.id, npc_role=self.role,
        building_id=building.id,
        file_path=building.file_path,
        question=question,
        local_attempts=local_attempts,
        context_snippet=source[:800],
        reason=reason,
        language=language,
        file_type=building.building_type,
        city_context={
            "cc":       building.metadata.get("cc", 0),
            "coverage": building.metadata.get("coverage", 0),
        }
    )

    response = await self.escalation.escalate(req)

    await self._emit("escalation_complete", {
        "npc_id":     self.id,
        "building":   building.id,
        "answer":     response.answer[:120],
        "provider":   response.provider_used,
        "from_cache": response.from_cache,
        "confidence": response.confidence,
    })
```

```
    # Use action items as guided prompt for local Ollama execution
    result = await self._execute_with_guidance(building, response)
    result["escalated"] = True
    result["used_kb"]   = response.from_cache

    # Mark knowledge base entry useful if we found bugs
    if result.get("bugs") and response.cache_entry_id:
        await self.escalation.kb.mark_useful(
            response.cache_entry_id,
            produced_bugs=len(result["bugs"])
        )

    return result

async def _execute_with_guidance(self, building, guidance) -> dict:
    action_steps = "\n".join(f"- {s}" for s in guidance.action_items)
    source = self._read_source(building.file_path)

    raw = await self.ollama.generate(
        model="deepseek-coder-v2:16b",
        system=f"""Execute this exact testing plan:
```

{action_steps} Output ONLY a JSON array of test cases. Each: {{test_type, description, input_data, expected, why}} No markdown. No explanation.”””, prompt=f”File: {building.file_path}\n\n `\n{source[:2000]}\n` ”, temperature=0.1) return self._parse_tests(raw, building)

```
def _choose_target(self, city_state, rank: str):
    buildings = [b for b in city_state.get_buildings()
                 if b.file_path not in self.memory]
    if not buildings:
        self.memory.clear()
        buildings = city_state.get_buildings()

    def score(b):
        base = b.metadata.get("cc", 1) * (b.roads_out + 1)
        # Senior+ agents prioritize untested high-complexity files
        if rank in ("senior", "principal"):
            base *= (1.5 if b.metadata.get("coverage", 1) < 0.2 else 1.0)
        return base + self._role_bonus(b)

    return max(buildings, key=score, default=None)

def _role_bonus(self, b) -> float:
    """Override in subclasses for role-specific targeting."""
    return 0.0

def _is_trivial(self, result: dict) -> bool:
    tests = result.get("tests", [])
    if not tests: return True
    trivial = sum(1 for t in tests
                  if t.get("input_data") in [None, "", [], {}])
    return trivial == len(tests)

def _failure_question(self, building, attempts) -> str:
    return (
        f"I'm testing {building.file_path} "
        f"(CC={building.metadata.get('cc',0):.1f}, "
        f"lang={building.metadata.get('language','?')}) "
        f"but 3 local attempts only produced trivial tests. "
        f"What specific non-trivial test cases would exercise "
        f"the real logic paths in this file?"
    )

def _read_source(self, file_path: str) -> str:
    try:
        return open(file_path, encoding="utf-8", errors="ignore").read()
    except:
        return ""

def _parse_tests(self, raw: str, building) -> dict:
```

```
    try:
        clean = raw.strip().lstrip("```json").rstrip("```").strip()
        tests = json.loads(clean)
        bugs = [t for t in tests if not t.get("expected_pass", True)]
        return {"success": True, "tests": tests, "bugs": bugs}
    except:
        return {"success": False, "tests": [], "bugs": []}

async def _emit(self, event: str, data: dict):
    await self.bus.emit(event, data)

async def _local_attempt(self, building) -> dict:
    raise NotImplementedError("Implement in subclass")
```

SECTION 9 — GODOT SYSTEMS (SEASON, TIMELINE, BULLETIN BOARD, DIALOGUE)

——— scripts/systems/SeasonSystem.gd ———

extends Node

Drives city visual season from overall health score.

Health score comes from backend via WebSocket.

@onready var canvas_mod : CanvasModulate = $CanvasModulate @onready var season_part : GPUParticles2D = $SeasonParticles @onready var ambient_audio: AudioStreamPlayer = $AmbientAudio

var current_season : String = “summer” var target_season : String = “summer” var health_score : float = 1.0

const SEASON_COLORS = { “summer”: Color(1.00, 1.00, 0.95), “spring”: Color(0.95, 1.00, 0.92), “autumn”: Color(0.90, 0.82, 0.70), “winter”: Color(0.75, 0.80, 0.95), }

const SEASON_AMBIENT = { “summer”: “res://assets/audio/ambient/city_summer.ogg”, “spring”: “res://assets/audio/ambient/city_spring.ogg”, “autumn”: “res://assets/audio/ambient/city_autumn.ogg”, “winter”: “res://assets/audio/ambient/city_winter.ogg”, }

const HEALTH_TO_SEASON = [[0.80, “summer”], [0.60, “spring”], [0.40, “autumn”], [0.00, “winter”],]

func _ready(): EventBus.metrics_updated.connect(_on_metrics)

func _on_metrics(data: Dictionary): health_score = data.get(“health_score”, 1.0) var new_season = “winter” for pair in HEALTH_TO_SEASON: if health_score >= pair[0]: new_season = pair[1] break if new_season != target_season: target_season = new_season _transition_to(new_season)

func _transition_to(season: String): current_season = season var target_color = SEASON_COLORS[season] var tween = create_tween() tween.tween_property(canvas_mod, “color”, target_color, 60.0)

```
# Season-specific particles
match season:
    "autumn":
        season_part.process_material.set("color", Color(0.8, 0.5, 0.2))
        season_part.emitting = true     # falling leaves
    "winter":
        season_part.process_material.set("color", Color(1.0, 1.0, 1.0, 0.8))
        season_part.emitting = true     # snow
    _:
        season_part.emitting = false

# Citizen density reacts to season
var density = {"summer":1.0, "spring":0.8, "autumn":0.5, "winter":0.2}
EventBus.emit_signal("set_citizen_density", density[season])

# Swap ambient audio
if SEASON_AMBIENT.has(season):
    ambient_audio.stream = load(SEASON_AMBIENT[season])
    ambient_audio.play()
```

——— scripts/systems/GitTimeline.gd ———

extends CanvasLayer

Press T to open. Drag slider to scrub through git history.

City transforms to reflect the codebase at that point in time.

@onready var timeline_slider : HSlider = $Panel/Slider @onready var date_label : Label = $Panel/DateLabel @onready var close_btn : Button = $Panel/CloseBtn

var commit_list : Array = [] # [{hash, date, message, stats}] var is_open : bool = false

func _ready(): hide() timeline_slider.value_changed.connect(_on_scrub) close_btn.pressed.connect(_close) EventBus.city_built.connect(_on_city_built)

func _input(event): if event is InputEventKey and event.pressed: if event.keycode == KEY_T: if is_open: _close() else: _open()

func _on_city_built(_count): BackendBridge.send({"type": "request_git_timeline"})

func receive_timeline(data: Dictionary): commit_list = data.get("commits", []) if commit_list.is_empty(): return timeline_slider.min_value = 0 timeline_slider.max_value = commit_list.size() - 1 timeline_slider.value = commit_list.size() - 1 # latest

func _on_scrub(value: float): var idx = int(value) var commit = commit_list[idx] date_label.text = "%s — %s" % [commit.get("date","?"), commit.get("message","?")[:50]] # Ask backend to re-render city at this commit BackendBridge.send({ "type": "render_at_commit", "commit_hash": commit.get("hash","") })

func _open(): show() is_open = true

func _close(): hide() is_open = false # Return to HEAD BackendBridge.send({"type": "render_at_commit", "commit_hash": "HEAD"})

——— scripts/world/BulletinBoard.gd ———————————————————————————————————————

extends Node2D

Physical board in Developer Tower + each district.

NPCs walk up and "pin" findings. Player can read them.

Fixing code lets the NPC walk back and remove the note.

signal note_clicked(note_data: Dictionary)

@onready var notes_container : Node2D = $Notes @export var max_notes : int = 12

var posted_notes : Array = []

func post_note(npc_id: String, npc_role: String, building_id: String, severity: String, message: String): if posted_notes.size() >= max_notes: _remove_oldest_note()

```
var note = _create_note_sprite(npc_role, severity, message)
var pos  = _find_free_slot()
note.position = pos
notes_container.add_child(note)
posted_notes.append({
    "node": note, "building_id": building_id,
    "npc_id": npc_id, "message": message, "severity": severity
})

# Arrival animation: note flies in from off-screen
note.position.y -= 30
var tween = create_tween()
tween.tween_property(note, "position:y", pos.y, 0.3)\
     .set_trans(Tween.TRANS_BACK)
```

func remove_note(building_id: String): for i in range(posted_notes.size() - 1, -1, -1): var n = posted_notes[i] if n["building_id"] == building_id: # NPC pulls note off animation var tween = create_tween() tween.tween_property(n["node"], "position:y", n["node"].position.y - 30, 0.2) tween.tween_callback(n["node"].queue_free) posted_notes.remove_at(i)

func _create_note_sprite(role: String, severity: String, message: String) -> Node2D: var note = Node2D.new() var rect = ColorRect.new() rect.size = Vector2(28, 20) rect.color = _severity_color(severity).darkened(0.2) note.add_child(rect)

```
var lbl = Label.new()
lbl.text = message[:24] + ("..." if message.size() > 24 else "")
lbl.add_theme_font_size_override("font_size", 5)
lbl.position = Vector2(2, 2)
note.add_child(lbl)

var area = Area2D.new()
var col  = CollisionShape2D.new()
col.shape = RectangleShape2D.new()
col.shape.size = Vector2(28, 20)
area.add_child(col)
area.input_event.connect(
    func(_vp, event, _s):
        if event is InputEventMouseButton and event.pressed:
            emit_signal("note_clicked",
                {"building_id": "", "message": message})
)
note.add_child(area)
return note
```

func _severity_color(sev: String) -> Color: match sev: “critical”: return Color(1.0, 0.2, 0.2) “high”: return Color(1.0, 0.6, 0.0) “medium”: return Color(1.0, 1.0, 0.2) _: return Color(0.8, 0.9, 1.0)

func _find_free_slot() -> Vector2: var cols = 3 var idx = posted_notes.size() return Vector2((idx % cols) * 32, (idx / cols) * 24)

func _remove_oldest_note(): if posted_notes.is_empty(): return posted_notes[0][“node”].queue_free() posted_notes.remove_at(0)

——— scripts/world/NPCAgent.gd (dialogue section) ———————————————————————

Called when player presses E near NPC

func interact(): $DialogueBox.show() $DialogueBox/NameLabel.text = “%s #%s [%s]” % [role.capitalize(), npc_id, reputation_rank.to_upper()] $DialogueBox/StatusLabel.text = “At: %s | State: %s” % [current_building, current_state] $DialogueBox/InputField.grab_focus()

func _on_question_submitted(text: String): if text.strip_edges().is_empty(): return $DialogueBox/InputField.editable = false $DialogueBox/Thinking.show() BackendBridge.send({ “type”: “player_asks_npc”, “npc_id”: npc_id, “question”: text, })

func receive_dialogue_response(data: Dictionary): $DialogueBox/Thinking.hide() $DialogueBox/InputField.editable = true

```
var src = data.get("source", "local")
$DialogueBox/SourceBadge.text = (
    "💾 local" if src == "local" else
    "📚 knowledge base" if "knowledge_base" in src else
    "🌐 " + src
)
$DialogueBox/SourceBadge.modulate = (
    Color.CYAN    if src == "local" else
    Color.YELLOW  if "knowledge_base" in src else
    Color(1,0.7,0)
)

$DialogueBox/AnswerLabel.text = data.get("text", "")

if data.get("offer_escalation", false):
    $DialogueBox/EscalateBox.show()
    $DialogueBox/EscalateLabel.text = data.get("escalation_prompt","")
    $DialogueBox/YesBtn.pressed.connect(
        func(): _confirm_escalation(data), CONNECT_ONE_SHOT
    )
    $DialogueBox/NoBtn.pressed.connect(
        func(): $DialogueBox/EscalateBox.hide(), CONNECT_ONE_SHOT
    )
```

func _confirm_escalation(data: Dictionary): $DialogueBox/EscalateBox.hide() $DialogueBox/Thinking.show() _show_consulting_fx() BackendBridge.send({ “type”: “escalation_confirm”, “npc_id”: npc_id, “question”: data.get(“question_for_escalation”, “”), })

func _show_consulting_fx(): $ThoughtBubble.text = "📡" $ThoughtBubble.visible = true _consulting_tween = create_tween().set_loops() _consulting_tween.tween_property($ThoughtBubble, "modulate:a", 0.2, 0.4) _consulting_tween.tween_property($ThoughtBubble, "modulate:a", 1.0, 0.4)

SECTION 10 — WEBSOCKET PROTOCOL (COMPLETE + NEW MESSAGES)

All messages JSON. ← = Godot→Backend. → = Backend→Godot.

CITY LIFECYCLE: ← { "type":"load_repo", "path":"C:/projects/myapp" } → { "type":"status", "message":"...", "progress":0.0–1.0 } → { "type": "city_ready", "data":{...CityLayout...} } ← { "type":"reload_city" } → { "type":"city_patch", "data":{"changed_buildings":[...]} }

SEASON + HEALTH: → { "type":"health_update", "score":0.73, "season":"spring" }

GIT TIMELINE: ← { "type":"request_git_timeline" } → { "type":"git_timeline", "commits":[{"hash","date","message","stats"}] } ← { "type": "render_at_commit", "commit_hash":"abc123" } → { "type":"city_ready", "data":{...city at that commit...} }

AGENTS: ← { "type":"spawn_agents", "count":8, "roles":["qa","fuzzer",...] } → { "type":"agent_spawned", "npc_id":"npc_001","role":"qa","x":0, "y":0,"rank":"junior" } ← { "type":"stop_agents" }

NPC MOVEMENT + STATE: → { "type":"npc_move", "npc_id":"npc_001","target":"src_api_py","x":42,"y":18 } → { "type":"npc_state", "npc_id": "npc_001","state":"testing|consulting|returning|idle", "building":"src_api_py","reason":"..." } → { "type":"npc_speed_update", "npc_id":"npc_001", "speed":100,"rank":"senior" } → { "type":"npc_report", "npc_id":"npc_001","result":{...} }

ESCALATION EVENTS: → { "type":"escalation_start", "npc_id":"npc_001","building":"...","reason":"..." } → { "type":"escalation_complete", "npc_id": "npc_001","building":"...", "answer":"...","provider":"claude|knowledge_base", "from_cache":false,"confidence":"high" }

BULLETIN BOARD: → { "type":"post_note", "npc_id":"...","building":"...","severity":"high","message":"..." } → { "type":"remove_note","building":"..." }

BUG + METRIC EVENTS: → { "type":"bug_found", "building_id":"...","severity":"critical|high|medium|low", "test_name":"...","error":"...", "stack_trace":"..." } → { "type":"metric_update", "building_id":"...","metric":"high_cpu|memory_leak|high_latency|crash|no_tests","value":0.0–1.0 } → { "type":"test_passed", "building_id":"...","coverage":0.94 }

PLAYER ↔ NPC DIALOGUE: ← { "type":"player_asks_npc", "npc_id":"npc_001","question":"..." } → { "type":"npc_dialogue", "npc_id":"npc_001", "text":"...","source":"local|knowledge_base|claude", "confidence":0.82, "offer_escalation":false } ← { "type":"escalation_confirm","npc_id": "npc_001","question":"..." } → { "type":"npc_dialogue", "npc_id":"npc_001", "text":"...","source":"claude","confidence":"high", "action_items": ["...","..."] }

KNOWLEDGE BASE STATUS (shown in Tower UI): → { "type":"kb_stats", "total_entries":47,"total_cache_hits":312,"hit_rate":0.94 }

BUILDING INSPECTION: ← { "type":"inspect_building", "building_id":"..." } → { "type":"building_detail", "building_id":"...","file_path":"...","loc":342, "complexity":18.4,"functions":[...],"classes":[...], "test_coverage":0.41,"commit_count":87, "last_commit":"2024-01-15","recent_bugs":[...] }

SECTION 11 — CITY GENERATION ALGORITHM

PHASE 1 — PARSE Scan repo recursively (skip: .git, node_modules, **pycache**, .venv, files >500KB, binary files) For each file: detect language (extension + pygments content sniff) classify: source | test | config | api | database measure: LOC (non-blank non-comment), CC (radon), MI (radon) extract: classes, functions, import strings resolve imports to relative file paths where possible git blame: commit count, first/last date (graceful no-git fallback)

PHASE 2 — CLUSTER Build NetworkX DiGraph: nodes=files, edges=import dependencies Community detection: folder structure primary Sub-divide large folders (>20 files) with python-louvain Sort communities: total LOC descending (largest = most central)

PHASE 3 — SPIRAL DISTRICT PLACEMENT Place community[0] at grid (0, 0) Remaining communities spiral outward: ring 1: right(70), down(70), left(70), up(70) ring 2: 8 positions at distance 140 ring N: 4N positions at distance N*70 Each district: grid-pack buildings L→R, wrap at 56 tile width 2-tile gap between buildings, 4-tile district border sidewalk tiles drawn along building fronts

PHASE 4 — BUILDINGS height = clamp(LOC / 50, 1, 10) floors width = clamp(int(CC / 4) + 1, 2, 6) tiles visual_age: commits < 5 → "new" commits < 30 → "modern" commits < 150 → "aged" commits >= 150 → "ancient" building_type → sprite_key lookup table (see Section 3 assets)

PHASE 5 — ROADS (A* on tile grid) Mark building tiles as blocked For each dependency edge: same district → street (width 1 tile) adjacent dist. → avenue (width 2 tiles) distant dist. → highway (width 3 tiles) target is api → highway regardless of distance Path follows district borders where possible Add traffic jam particles on roads of latency-flagged buildings

PHASE 6 — SPECIAL BUILDINGS Developer Tower: exact center (0,0), 6×8 tiles, always rendered City Hall: center of config-file cluster, admin sprite Central Station: highest in-degree API file, at district edge Bank District: all database files within 10 tiles of each other

PHASE 7 — ATMOSPHERE 5% road tiles → lamp post decoration node 2% empty tiles → tree / park tile 1 citizen per 20 road tiles (decorative, path-follow only) 1 bulletin board per district (placed at district entrance) All particles dormant until metric events fire

PHASE 8 — HEALTH + SEASON Compute initial health score from analysis results Set opening season: city loads into correct visual state immediately

GIT TIMELINE (backend, for scrubbing): git log –format="%H|%ai|%s" → parse into commit list For each commit: store summary stats (files changed, insertions, deletions) On render_at_commit: git checkout –detach → re-parse → city_ready On HEAD restore: git checkout HEAD

SECTION 12 — GODOT VISUAL DESIGN SPECIFICATION

AESTHETIC: Cozy simulation by day. Noir city by night. Melancholy ghost town in winter. Alive in summer.

TILE SIZE: 16×16 pixels. All sprites on 16px grid.

BUILDING COLORS (base before age shader): office: #c4a882 walls / #6b8fad windows tower: #7a9ab5 glass / #ffffff highlights lab: #d4f0e8 mint / #00ff88 neon accent bank: #8b7355 marble / #d4af37 gold trim station: #b5785a brick / #f0d080 warm yellow admin: #a09070 gray / #8b0000 dark red flag warehouse: #7a6a5a brown / #c0c0c0 metal roof

AGE SHADERS (gdshader — write all four): new: glow_intensity 0.4, crisp edges, vivid color modern: standard, saturation +10% aged: desaturate 25%, yellow tint 0.08, dark marks texture ancient: desaturate 50%, sepia 0.3, crumble edge noise

PARTICLE SYSTEMS (GPUParticles2D — all GPU-driven): fire: orange/yellow upward burst, 60 particles, 8px spread smoke: gray slow upward drift, 20 particles, 40px spread flood: blue sparkle rising from base, 30 particles sparkle: green/gold confetti (test passed), 15 particles, one-shot collapse: gray debris outward burst, 80 particles, one-shot leaves: orange/brown falling diagonally (autumn season) snow: white slow falling, wide spread (winter season) consulting: blue circular pulse from NPC (escalation in progress)

RANK VISUALS (NPC sprites — use Kenney roguelike pack sprites): junior: standard sprite, no badge mid: yellow star (⭐) floating above, speed 90px/s senior: gold badge + scale 1.1x, speed 100px/s principal: cyan glow shader + unique tint, speed 110px/s

LIGHTING (Godot 2D global illumination): Each building: PointLight2D at entrance, energy 0.3, warm yellow Night: window glow shader activates (parameter driven by time) Street lamps: PointLight2D energy 0.8 warm white Fire: PointLight2D energy 2.0, red/orange, flickering tween Developer Tower: always cyan ambient glow Consulting NPC: blue soft PointLight2D pulses during escalation

HUD LAYOUT: Top-left: minimap (160×120px, auto-update every 5s) Top-right: health bar (color reflects season), live metrics strip Bottom-left: knowledge base stats: "📚 47 entries / 94% hit rate" Bottom-center: building name tooltip on hover Bottom-right: time of day / season icon / git commit (if timeline active) Center: loading progress (city generation only)

SEASON OVERLAY (CanvasLayer, always on top at low alpha): summer: no overlay spring: faint green tint, occasional pollen particles autumn: warm sepia tint 0.15, falling leaves particles winter: cool blue tint 0.25, snow particles, reduced ambient light

BULLETIN BOARD (per district + Developer Tower): Physical wooden board sprite from Kenney Tiny Town assets Notes = small colored rectangles with truncated text labels New note: flies in from top with spring bounce tween Fixed note: NPC walks up, note flies off upward and disappears Click note: camera flies to relevant building, Inspector opens

CAMERA: WASD + arrow keys: movement (800px/s, smooth acceleration) Mouse wheel: zoom 0.25×–4× with smooth lerp Middle mouse drag: pan F: fly to Developer Tower T: toggle Git Timeline panel M: toggle minimap E: interact with nearest NPC (within 32px) Click building: smooth zoom toward it + open Inspector

---

SECTION 13 — PERFORMANCE ARCHITECTURE

Large repos (>500 files): → Stream city_ready in chunks of 50 buildings → Godot renders each chunk; player can explore before full load

LLM concurrency (RTX 1070): → asyncio.Semaphore(8) in OllamaClient — enforced globally → Priority queue: high-CC low-coverage buildings first → Fast model (6.7b) for dialogue; primary model (16b) for test gen

NPCs (50 visible, 8 active LLM): → Godot: 50 CharacterBody2D — visual only, lightweight → Python: 8 async coroutines — LLM + test execution → Knowledge base cache means many "escalations" cost zero API calls

File reload (Watchdog → 2s update): → 500ms debounce batch window → Re-parse only changed files → Send city_patch with delta only → Godot updates only affected building nodes

60fps rendering: → GPUParticles2D (zero CPU cost) → CanvasLight only in camera viewport → NPCs outside viewport: physics paused, animation skipped → Single TileMap with multiple layers → VisibilityNotifier2D disables particles when off-screen

Knowledge base (growing over time): → SQLite is lightweight, query time < 1ms for 10,000 entries → Index on (problem_type, language) covers

95% of queries → Context hash lookup is O(1) for exact matches

---

SECTION 14 — WINDOWS BATCH FILES

——— install.bat ———
@echo off echo Installing Software City...

REM Install prerequisites via winget winget install -e –id Python.Python.3.11 –accept-source-agreements winget install -e –id GodotEngine.GodotEngine –accept-source-agreements winget install -e –id Ollama.Ollama –accept-source-agreements

REM Python backend deps cd backend pip install -r requirements.txt cd ..

REM Playwright browser playwright install chromium

REM Download all free assets python tools/download_assets.py

REM Create data directory + databases mkdir data 2>nul python -c "import asyncio; from ai.knowledge_base import KnowledgeBase; asyncio.run(KnowledgeBase().initialize())" python -c "import asyncio; from ai.reputation import ReputationSystem; asyncio.run(ReputationSystem().initialize())"

echo. echo Installation complete! echo Next step: run pull_models.bat to download AI models pause

——— pull_models.bat ———
@echo off echo Pulling AI models (this may take 20-40 minutes)... start /b "" ollama serve timeout /t 3 /nobreak > nul

ollama pull deepseek-coder-v2:16b ollama pull deepseek-coder:6.7b

echo. echo Models ready. Run launch_software_city.bat to start. pause

——— launch_software_city.bat ——— @echo off echo Starting Software City...

REM Load API keys from .env if present if exist .env (for /f "tokens=1,2 delims==" %%a in (.env) do set %%a=%%b)

REM Start Ollama tasklist /fi "imagename eq ollama.exe" | find "ollama.exe" > nul if errorlevel 1 (echo Starting Ollama... start /b "" ollama serve timeout /t 3 /nobreak > nul)

REM Start Python backend echo Starting backend... start /b "" python -m uvicorn backend.main:app –port 8765 –log-level warning

timeout /t 2 /nobreak > nul

REM Launch Godot echo Launching Software City... start "" godot –path godot/

echo. echo Software City is running. echo Close this window to stop all services. pause

REM Cleanup on exit taskkill /f /im uvicorn.exe 2>nul taskkill /f /im godot.exe 2>nul

——— .env template (rename to .env, never commit) ——— ESCALATION_PROVIDER=claude ANTHROPIC_API_KEY=sk-ant-your-key-here GROQ_API_KEY=your-groq-key-here TOGETHER_API_KEY=your-together-key-here OPENAI_API_KEY=your-openai-key-here

——— backend/requirements.txt ———
fastapi==0.109.0 uvicorn[standard]==0.27.0 websockets==12.0 pydantic==2.5.3 radon==6.0.1 gitpython==3.1.41 tree-sitter==0.21.3 tree-sitter-languages==1.10.2 networkx==3.2.1 pygments==2.17.2 psutil==5.9.8 watchdog==3.0.0 httpx==0.26.0 hypothesis==6.98.1 pytest==7.4.4 pytest-cov==4.1.0 pytest-asyncio==0.23.3 playwright==1.41.1 locust==2.20.1 pillow==10.2.0 python-louvain==0.16 aiosqlite==0.19.0

SECTION 15 — CLAUDE.md (ROOT FILE, READ BY CLAUDE CODE)

SOFTWARE CITY — Context for Claude Code

What This Is

A system that renders any code repository as a procedurally generated, explorable pixel-art city. Autonomous local AI agents (Ollama NPCs) walk around testing the code. They escalate to external AI only when stuck locally. Every escalation answer is saved permanently to a local knowledge base — the city grows smarter over time. Code health drives a visible season system. Git history can be scrubbed to watch the city evolve through time.

Stack

- Godot 4.3 (game, GDScript)
- Python 3.11 + FastAPI + WebSocket (backend, port 8765)
- Ollama local (deepseek-coder-v2:16b primary, 6.7b for dialogue)
- External AI escalation: Claude/Groq/Together (only when NPCs stuck)
- SQLite: knowledge_base.db, npc_reputation.db, bug_reports.db

Key Architecture Rules

- NPCs try local Ollama FIRST (max 3 attempts)
- Knowledge base checked BEFORE any external API call
- External AI only called on genuine failure or player request
- Every external AI answer saved to knowledge_base.db permanently
- Max 8 simultaneous Ollama calls (asyncio.Semaphore enforced)
- City season derived from real-time health score (coverage + bugs + CC)
- Git timeline scrubbing: city transforms to any historical commit

File Map

- ai/knowledge_base.py → learning system (critical)
- ai/escalation_engine.py → local fail → knowledge search → external AI
- ai/reputation.py → NPC accuracy tracking + rank promotion
- ai/agents/base_agent.py → unified agent loop with all systems wired
- backend/core/health_scorer.py → health score → season
- godot/scripts/systems/SeasonSystem.gd → visual season
- godot/scripts/systems/GitTimeline.gd → time scrubbing
- godot/scripts/world/BulletinBoard.gd → NPC posting board

Current State

Read PROGRESS.md for session history.

SECTION 16 — MILESTONE BUILD ORDER

M1 — SKELETON (2h): install.bat works · FastAPI /health · Godot connects M2 — THE MAP (4h): repo parsed · rectangles in Godot · WASD works M3 — ALIVE (6h): Kenney assets · roads · day/night · citizens · inspector M4 — TESTERS (8h): Ollama generating tests · NPCs walking · reports in tower M5 — ESCALATION(4h): knowledge_base.db · escalation flow · dialogue with offer M6 — LEARNING (2h): knowledge base grows · cache hits logged in HUD M7 — REPUTATION(2h): NPC rank badges · speed differences visible M8 — BULLETIN (2h): boards in each district · notes post/remove M9 — SEASONS (4h): health score · season transitions · seasonal particles M10— TIMELINE (4h): git scrubber · city transforms through history M11— SHIP IT (4h): launch.bat · Godot .exe export · README · fresh Windows test

---

SECTION 17 — FREE BUILDERS TO USE FOR THIS PROJECT

Use all of these in combination. Each handles a different layer.

GAME LAYER: Godot 4.3 godotengine.org — the game engine Ziva (Godot plugin) itch.io/t/6002362/ziva — AI agent inside Godot editor free tier available Godot MCP github.com/tomyud1/godot-mcp — Claude/Cursor talks directly to Godot editor via MCP (MIT license, free)

CODING WITH AI: Claude Code claude.ai — VS Code extension + this chat Cursor cursor.sh — AI-native VS Code fork, free tier Windsurf codeium.com/windsurf — free AI IDE, Codeium model Aider aider.chat — terminal AI pair programmer works with any local/API model completely free + open source Continue.dev continue.dev — open source AI IDE extension works with Ollama locally

LOCAL AI: Ollama ollama.com — local model server (the core) LM Studio lmstudio.ai — GUI for local models, free Jan jan.ai — another local model GUI, free

PIPELINE / AUTOMATION: n8n n8n.io — free self-hosted automation can trigger city reloads, post Slack/Discord alerts Langflow langflow.org — visual LangChain builder, free open source, local

VERSION CONTROL + CI: GitHub github.com — free for public + private repos Gitea gitea.io — self-hosted GitHub alternative Act github.com/nektos/act — run GitHub Actions locally free

PIXEL ART TOOLS: Libresprite libresprite.github.io — free Aseprite fork make custom building sprites Pixelorama orama.itch.io/pixelorama — free open source pixel editor Godot-based, exports to .png

MONITORING (optional dashboard): Grafana grafana.com — free self-hosted dashboards visualize your metrics Prometheus prometheus.io — free metrics collection

OPTIMAL WORKFLOW COMBINING THESE:

1. Godot 4.3 + Godot MCP → Claude Code (this chat) writes GDScript directly into your Godot scene via MCP. No copy-paste.
2. Cursor or Windsurf → write Python backend with AI autocomplete
3. Aider in terminal → quick edits and bug fixes across multiple files
4. Ollama + LM Studio → manage and test your local models visually
5. n8n → automate: on git push → reload city → post report to Discord
6. Libresprite → customize Kenney building sprites for your own style

---

SECTION 18 — THE DESIGN PHILOSOPHY

This project is a mirror.

When you walk through Software City, you are walking through your own thinking — made visible, given weight, given weather.

The complexity of a class becomes a maze you must navigate. The dependencies of your architecture become roads that connect or isolate whole districts from each other. The health of your codebase is the season outside your window.

A file nobody touches becomes an abandoned building, dark-windowed, overgrown, drifting toward winter.

A file everyone imports becomes a glowing train station at the center of the district — always busy, always lit, always connected to everything.

The NPCs are your workforce. They never sleep. They never complain. When they hit something they don't understand, they don't guess. They ask for help. And they remember the answer forever.

Every escalation makes the city smarter. Every fixed bug warms the season. Every new test lights a window that was dark.

The city is the code. The code is the city. Build it and see yourself clearly.

NOW BUILD ALL OF IT

Write every file. Write every function. Write every shader. Every module is fully specified. Every algorithm is written out. Every asset has a download URL. Every batch file has its content.

The knowledge base makes every escalation permanent. The reputation system makes every NPC's career visible. The season system makes code health emotionally legible. The git timeline makes history walkable. The bulletin board makes findings social.

Start with install.bat. End with a city that knows your code better every day.